

Week 5: Testing, Debugging and Final Integration

System Setup

The final week focused on testing the complete RESQAI backend.

The required dependencies were installed separately for the Node.js backend and Python ML service.

For the backend:

```
```bash
```

```
cd backend
```

```
npm install
```

For the ML service:

```
```bash
```

```
cd ml-service
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

The trained BERTweet model was configured inside:

```
```text
```

```
ml-service/models/bertweet_model/
```

Environment variables were also configured for MongoDB, JWT authentication, the ML service URL, the co

```
Starting the System
```

The complete system requires three services:

```
```text
```

```
Terminal 1 → MongoDB
```

```
Terminal 2 → ML Service
```

```
Terminal 3 → Node.js Backend
```

Health endpoints were used to verify that the services were running:

```
```bash
```

```
curl http://localhost:5000/health
```

```
curl http://localhost:8000/health
```

```
API Testing
```

The authentication workflow was tested by registering users with different roles:

```
```text
```

```
Reporter
```

```
Admin
```

```
Rescue Team
```

The reporter was then logged in and provided with a JWT token.

An incident was subsequently created using the token:

```
```text
```

```
"Building collapsed! People trapped!"
```

```
along with location information.
```

```
The created incidents could then be retrieved using the authenticated API.
```

```
Troubleshooting
```

```
Common issues were identified and documented.
```

```
ML Service Issues
```

```
Possible causes included:
```

- Missing BERTweet model
- Missing Gemini API key
- Python import errors

```
Backend Issues
```

```
Possible causes included:
```

- MongoDB not running
- Port already in use
- ML service unavailable

```
Classification Issues
```

```
The ML service health endpoint and direct `/classify` endpoint could be tested independently to identify issues.
```

```
Documentation
```

```
Documentation was finalized for:
```

```
```text
```

```
BACKEND_SETUP.md
```

```
API_EXAMPLES.md
```

```
backend/README.md
```

```
ml-service/README.md
```

```
PROJECT_SUMMARY.md
```

These documents cover setup, API testing, architecture and troubleshooting.

Final Outcome

At the end of Week 5, the RESQAI backend contained:

- Complete Node.js backend
- Python ML service
- BERTweet classification
- Gemini fallback

- MongoDB persistence
- JWT authentication
- Role-based access control
- Incident lifecycle management
- Rescue-team assignment
- RESTful APIs
- Input validation and security mechanisms

The implementation summary reports **31 files created, 3,000+ lines of code and 15+ API endpoints**, with the system marked complete and ready to use.